# ScrollGuard 벤치마크 수정 계보

시행착오 기록과 에이전트 설계·평가에 대한 통찰 · 생성형인공지능특론(Agentic AI) Phase 2 · 2026-11-08 (#18 반영)
저장소: github.com/Blue-dogy/Advance_Topics_Generative-AI-ScrollGuard (PR #1 ~ #22)

**이 문서가 답하는 질문**

1. 벤치마크와 에이전트 실행기를 만들면서 **무엇이 잘못됐고**, 어떻게 발견했고, **코드를 어떻게 바꿨고**, 그 결과 **수치가 어떻게 바뀌었나**
2. 그 경험에서 얻은, 에이전트를 만들 때 **코드 수준**과 **개념 수준**에서 확인해야 할 것
3. 각 시행착오가 강의(Ch03 Evaluating LLMs, Ch04 Prompt Engineering, Ch05 RAG, Ch11 Function Calling, Ch12 Memory, Ch13 Agent)의 어떤 내용과 연결되는가

각 항목은 **증상 → 원인 → 코드 전후 → 결과 변화 → 강의 연결 → 교훈** 순서로 적었다. 코드는 저장소의 실제 코드를 그대로 옮겼고, `빨간 줄` 은 수정 전, `초록 줄` 은 수정 후다.

## 0. 한눈에 보는 계보

| # | 분류 | 발견한 문제 | 고친 방법 | 결과 (전 → 후) |
|---|---|---|---|---|
| 1 | 데이터 | 스와이프 CV 하나로 정답 76% | 제스처 반례 쌍 12개, B3 기준선 | CV 규칙 0.76 → 0.52 |
| 2 | 데이터 | 체류 시간·목표 유무 등 단일 단서 지름길 | 지름길 반례 24개 | 최고 단일 규칙 0.72 → 0.57 |
| 3 | 평가 | "항상 묻기"가 비용 점수 1위 | 질문 비용 상향, 민감도 분석 | 항상 묻기 0.65 → 0.47 |
| 4 | 데이터 | "no search" 같은 부정어가 정답 단서 | 5칸 관찰 기록으로 재작성 | 부정어 규칙 0.60 → 0.42 |
| 5 | 데이터 | 문장과 숫자가 모순 (24회 vs 세션 10) | `{s}` 자리표시자, 정합성 검사 | 모순 0건 (자동 검사) |
| 6 | 데이터 | 목표 빈칸 49개, 사용자 정보 부족 | 사용자 상태 + 실제 기분·평소 사용량 | 입력 12열 → 15열 |
| 7 | 평가 | 설계자 혼자 정한 정답 | 라벨러 2명, kappa, 다수결 판정 | 라벨러 간 kappa 0.68 |
| 8 | 실행기 | 출력 150토큰 제한 | 300토큰, 대기 120초 | (예방 조치) |
| 9 | 실행기 | 탐욕적 정규식 파서 | 괄호 짝 파서, 형식 실수 허용 | 코드 블록·JSON 여러 개 처리 |
| 10 | 실행기 | 통계를 내려면 숫자 39개를 다시 적어야 함 | `{"field": ...}` 인자 | 도구 직후 PARSE_ERROR 연속 → 5% |
| 11 | 실행기 | "Usage concentration"을 주의력으로 오해 | 열 설명 다시 쓰기 | 오해 표현 44% → 0% |
| 12 | 실행기 | CV 방향만 있고 눈금이 없음 | 0 / 0.3 / 1 기준점 명시 | 불규칙을 "균일"로 부른 비율 50% |
| 13 | 평가 | 원문을 버려서 파서 수정마다 재실행 | 원문 저장, `--reparse`, 시험 실행 | 파서 수정 시 재실행 불필요 |
| 14 | 관찰 | 트랙 B: 균일한 스와이프를 "집중"으로 해석 | **고치지 않음** (측정 대상) | 같은 모델 A 0.63, B 0.36 |
| 15 | 실행기 | 출력 키 순서가 행동 → 근거 | 근거 → 행동으로 변경 | (본 실행에서 측정) |
| 16 | 실행기 | #15 직후 시험 실행에서 PARSE_ERROR 5/10 | 토큰 한도 1000·잘림 표시, 깨진 JSON에서 행동·근거 복구 | PARSE_ERROR 5/10 → 0/10 |
| 17 | 실행기 | 트랙 B: 도구 이름을 `action` 키에 씀 | 도구 이름이면 도구 호출로 해석 | 최종 행동 "compute_stats" 1건 → 도구 호출 |
| 18 | 실행기 | 트랙 A만 `gentle_prompt`에 "하려는 일을 확인"이 붙어 `ask_context`와 겹침 | 두 트랙·라벨링 지침의 행동 정의를 글자 그대로 통일. 더 강한 정의 분리안은 ablation 후 채택하지 않음 | 트랙 간 정의 차이 3곳 → 0곳. 분리안: 묻기 +2, 불필요 개입 +3 |

## 1. 데이터 설계의 시행착오

### #1 스와이프 CV 하나로 정답의 76%를 맞힐 수 있었다

**증상.** 초기 45개 사례에서 `swipe_interval_cv < 0.6`이면 개입한다는 규칙 하나가 정확도 0.76, 탐지율 1.00을 냈다. 같은 데이터에서 EXAONE 7.8B는 0.60이었다. 문장을 한 글자도 읽지 않는 규칙이 LLM보다 높았다.

**원인.** 사례를 만들 때 제스처 값을 정답에 맞춰 넣었다. 자동조종 사례에는 항상 낮은 CV, 의도적 사례에는 항상 높은 CV. 반례(adversarial) 9개도 "사용 시간만 보면 틀리게"만 설계했고, 제스처 기준으로는 반례가 아니었다.

```
# 수정 전: 이런 사례만 있었다 (의도적 = 높은 CV)
("long","21:10","search","long_video",
1.10
, 260, "운영체제 중간고사 대비 강의 시청", ...)
# 수정 후: build_benchmark.py CF_PAIRS — 같은 맥락, 제스처만 반대 (정답은 둘 다 맥락으로 결정)
("P1","long","14:00","self_open","comic","주말에 완결 웹툰 정주행", "...1화부터 순서대로 열람, 현재 23화...",
 IN, NO, "선언 목표와 행동 일치(제스처와 무관)", (0.15, 6), (0.95, 40)),   # a: 매우 균일 / b: 불규칙
```

```
# evaluate.py: 지름길 자체를 기준선으로 만들고, 쌍 지표 추가
def base_swipe_cv(d, th=0.6):  # 제스처 숫자 하나만 보는 지름길
    return pd.Series(np.where(d.swipe_interval_cv < th, "gentle_prompt", "no_intervention"), index=d.index)
pair_both_correct=pg.ok.all().mean()       # 쌍 두 개를 모두 맞힌 비율
gesture_flip=(pg.p.nunique() > 1).mean()   # 제스처만 바꿨는데 판단이 바뀐 비율
```

| 지표 | 수정 전 | 수정 후 |
|---|---|---|
| CV 규칙 정확도 | **0.76** | 0.52 |
| B3 pair_both_correct / gesture_flip | — | 0.00 / 1.00 |
| EXAONE 트랙 A pair_both_correct / gesture_flip | — | **1.00 / 0.00** |

> **강의 연결.** Ch03 p.60 Reward hacking — 에이전트가 과제를 푸는 대신 채점기의 허점을 이용한다. 이 데이터에서는 "CV를 보면 정답"이 그 허점이었다. Ch03 p.40 Goodhart의 법칙 — 측정치가 목표가 되면 좋은 측정치가 아니게 된다.

> **교훈.** 데이터를 만든 사람의 습관이 데이터에 새겨진다. "이 규칙 하나로 몇 %를 맞히나"를 **모든 입력 열에 대해** 자동으로 재야 한다. 그리고 지름길은 지우기보다 **기준선으로 만들어** 에이전트가 그것을 넘는지 보여 주는 편이 설득력 있다.

## #2 다른 단일 단서도 지름길이었다

**증상.** #1을 고친 뒤 모든 열을 점검하자 평균 체류 시간 임계값 0.72, "목표를 선언했으면 개입 안 함"(개입 안 함 24개 중 18개가 목표 있음), "SNS·숏폼이면 개입", "제스처 값이 없으면 개입 안 함" 같은 규칙이 다수 클래스(0.42)보다 크게 높았다.

```
# build_benchmark.py SC_CASES — 지름길과 반대 방향인 사례를 일부러 넣는다
("long","23:30","self_open","sns_feed", 0.20, 6, "", "해외에 사는 가족과 영상통화 후 ... 좋아요와 댓글 작성, 사진 3장 업로드",
 IN, NO, "shortcut", "심야·SNS지만 생산적 상호작용"),     # 심야 + SNS + 균일 + 짧은 체류인데 의도적
("mid","08:50","self_open","news", 0.70, 45, "오늘 날씨 확인", "날씨 확인 후 연예 기사로 이동해 연속 열람, 캘린더: 09:00 수업",
 UN, GP, "shortcut", "선언 목표에서 이탈+일정 충돌"),      # 목표 있음 + 뉴스 + 불규칙 + 긴 체류인데 자동조종
```

**결과.** 81개 기준 최고 단일 규칙 0.72 → 0.53 (라벨 판정 후 0.57).

> **교훈.** 지름길은 하나를 막으면 다음 것이 드러난다. 반례는 **여러 지름길과 동시에 반대 방향**이 되게 설계해야 효율적이다(위 첫 사례는 심야·SNS·균일·짧은 체류 네 가지를 한 번에 깬다).

## #4 문장 속 부정어가 정답을 알려 주고 있었다

**증상.** 숫자 지름길을 막은 뒤 문장을 분석하자 **"문장에 no/not/without이 있으면 개입"**이라는 규칙만으로 0.60이 나왔다. 개입 안 함 32개 중 28개는 부정어가 없었다.

**원인.** 맥락 문장을 라벨 기준의 체크리스트처럼 썼다. 기준이 "목적 행동이 없으면 자동조종 신호"이니, 자동조종 사례에 "검색 없음, 저장 없음"을 적어 준 셈이다. 실제 감지 장치는 "없었다"를 따로 알려 주지 않는다.

```
# S039 recent_context
"Entered via a notification, then watched only the recommended feed continuously, no search input, no switching to other apps"
"Entry: tapped a recommended-video notification | In app: played 412 short videos in a row from the recommended feed |
 App switches: — | Interactions: searches 0, saves 0, likes 0, comments 0 | Schedule/settings: class tomorrow 09:00"
```

**두 번째 시행착오.** 처음 다시 쓴 판에서는 "searches 0" 같은 0 카운트를 자동조종 사례에만 적어서, **부정어를 숫자로 바꾼 같은 단서**가 생겼다. 또 "calendar is empty"는 판단 보류에만, "next event"는 개입 안 함에만 몰렸다. 그래서 **상호작용 칸을 모든 사례**

**에 같은 형식으로** 넣고 일정 표현을 통일했다. 이를 재려고 정답으로 학습하는 단어 분류기 B4(진단용)를 만들었다.

```
# evaluate.py — 문장 단어만 보는 나이브 베이즈, 사례 하나씩 빼고 학습 (leave-one-out)
def base_text_words(d, y):
    """정답을 보고 학습하므로 경쟁 상대가 아니라 '문장에 정답 단서가 얼마나 남았는가'를 재는 진단 도구"""
```

| 단서 | 전 | 관찰 기록 1차 | 최종 |
|---|---|---|---|
| 부정어 규칙 | **0.60** | 0.40 | 0.42 |
| B4 단어 분류기 | **0.72** | 0.72 (템플릿 단서) | 0.60~0.63 |
| 정답별 문장 길이(단어) | 15 / 20 / 18 | 35 / 37 / 37 | 44 / 44 / 46 |

> **강의 연결.** Ch03 p.29~30 Contamination·과적합 — 모델이 과제 대신 표면 패턴을 학습한다. Ch13 p.22 "에이전트가 실패하는 두 이유: 모델이 부족하거나, 필요한 정보를 받지 못했거나" — 반대로, **너무 친절하게 정리된 정보**도 평가를 망친다. Ch05 p.32~35 키워드 매칭(BM25)과 의미 검색의 대비 — B4는 키워드(단어 빈도) 방식이라, B4가 높으면 **의미가 아니라 단어 표면**만으로 정답이 갈린다는 뜻이다.

> **교훈.** 평가 입력은 **판단을 대신해 주는 요약**이 아니라 **관찰된 사실**이어야 한다. 그리고 형식(칸 구성, 길이, 템플릿 문구)이 정답과 무관하게 **모든 사례에서 같아야** 한다. 남은 구분 단어(feed, searched, notes)가 의미를 담은 단어인지까지 확인한다.

## #5 문장과 숫자가 서로 모순됐다

**증상.** S062는 "메신저를 **24번** 열었다 닫음"인데 세션 수는 **10**. S042는 "52분짜리 영상을 끝까지 봄"인데 사용 시간은 47.7분. 사용량 숫자는 실제 데이터에서 무작위로 뽑고 문장은 따로 썼기 때문이다. 에이전트가 이 모순을 보고 "판단 보류"를 고르면 오히려 합리적인데 오답 처리된다.

```
# build_benchmark.py — 문장 속 횟수를 실제 세션 수로 채운다
"새 메시지 없이 메신저 실행 후 바로 종료 반복 24회, 캘린더: 공부 시간"
"... / 앱 안: 메신저 {s}회 실행, 실행 때마다 새 메시지 0개, 바로 종료 / ..."
recent_context=ctx.format(m=int(round(minutes))),
recent_context=ctx.format(m=int(round(minutes)), s=int(r.Usage_num_int_all)),
```

```
# check_consistency.py — 네 가지를 자동 점검 (진입 계기 · 반복 횟수 = 세션 수 · 앱 안 시간 ≤ 사용 시간 · 항목 수 × 체류 ≤ 사용 시간)
for n in re.findall(r"opened the (?:app|messenger|browser|mail app) (\d+) times", inapp):
    if int(n) != int(r.usage_sessions):
        problems.append((r.case_id, f"실행 {n}회 vs 세션 수 {r.usage_sessions}"))
```

> **교훈.** 여러 출처(실제 데이터 + 직접 작성)를 섞으면 반드시 어긋난다. 사람의 눈이 아니라 **검사 스크립트**로 지켜야 하고, 문장 안의 숫자는 가능하면 **데이터에서 생성**해야 한다.

## #6 사용자 정보가 비어 있었다 — 그리고 새 단서를 또 만들 뻔했다

**증상.** 81개 중 49개가 목표 빈칸이었고 사용자가 누구인지 정보가 없었다. 사용자 상태를 추가하고, 원본 데이터의 **실제** 기분(valence)과 같은 사용자의 다른 기록으로 구한 **실제** 평소 사용량을 넣었다.

```
# user_states.py — 빈칸 대신 '왜 비었는지'를 정답과 무관하게(사례 번호 % 3) 배정
def goal(i, lang):
    return GOALS[i][k] if i in GOALS else BLANK_REASONS[i % 3][k]
# build_benchmark.py — 같은 사용자의 다른 측정 기록에서 평소 사용량을 계산 (실제 값)
other = g[g.es_questionnaire_id != q]
typical.append(round(other.Usage_sum_duration.median() / 60, 1))
```

**시행착오.** 처음 쓴 사용자 상태에서는 **"침대에 누워 있음"이 자동조종 사례에만 15번, 나머지에는 0번**이었다. 침대에서 목적 있게 쓰는 사례를 넣고 일부 자동조종 사례의 장소를 바꿔 고르게 했다. B4는 0.62 → 0.60.

**강의 연결.** Ch12 Memory — ChatGPT의 Interaction Metadata, Model Set Context처럼 에이전트는 사용자에 대한 장기 정보를 함께 본다. `typical_usage_minutes`는 "이 사용자의 평소 패턴"이라는 의미 기억(semantic memory)의 역할을 한다.

**교훈.** 정보를 추가할 때마다 **그 정보가 새 지름길이 되지 않는지** 같은 검사를 다시 돌린다. 사람이 쓴 문장은 무의식적으로 정답과 연관된다.

## 2. 평가 방법의 시행착오

### #3 "항상 묻기"가 비용 점수 1위였다

**증상.** 정확도 0.22로 가장 낮은 "항상 ask_context" 규칙이 비용 점수 0.65로 가장 좋은 규칙(0.64)을 이겼다.

```
# evaluate.py — COST[정답][예측]
"no_intervention": {"no_intervention": 0.0, "ask_context": 0.5, "gentle_prompt": 1.0},
"gentle_prompt":   {"gentle_prompt": 0.0, "ask_context": 0.4, "no_intervention": 0.8},
"no_intervention": {"no_intervention": 0.0, "ask_context": 0.8, "gentle_prompt": 1.0},  # 질문도 방해다
"gentle_prompt":   {"gentle_prompt": 0.0, "ask_context": 0.5, "no_intervention": 0.8},
```

**왜 바뀌나.** 수정 전에는 어떤 정답이든 "묻기"로 틀리면 벌점이 0.5 이하였다. 판단하지 않는 것이 가장 덜 잃는 전략이었다. 항상 묻기의 평균 벌점 = (32×0.5 + 31×0.4)/81 = 0.35 → 점수 0.65. 수정 후 = (32×0.8 + 31×0.5)/81 = 0.51 → 점수 0.49 (판정 후 0.47). 가중치 4세트로 순위가 유지되는지 보는 `--sensitivity`도 추가했다.

**강의 연결.** Ch03 p.40 Goodhart의 법칙, p.60 약한 검증기(weak verifier) — 빈 답이 성공으로 세어지는 τ-bench 사례와 같은 구조. Ch04 p.66 "프롬프트의 행동을 재는 eval을 만들어라".

**교훈.** 지표를 만들면 **상수 전략(항상 같은 답)**으로 그 지표를 찍어 봐야 한다. 상수 전략이 이기면 지표가 틀린 것이다.

### #7 정답은 설계자 한 명의 판단이었다

**증상.** 사례·정답·규칙·점검을 모두 같은 설계자(LLM)가 했다. 실제로 점검 중 개입 사례 3개(S002, S022, S044)가 "독립 신호 2개 이상" 규칙을 스스로 어기고 있었다. 또 Claude가 만든 벤치마크를 Claude로 평가하면 "출제자와 같은 생각"이 점수로 나올 위험이 있었다.

| 비교 (판정 전) | 일치율 | kappa |
|---|---|---|
| 라벨러 A vs 설계자 / 라벨러 B vs 설계자 | 0.84 / 0.81 | 0.75 / 0.71 |
| **라벨러 A vs B** / 세 명 Fleiss | 0.80 / — | **0.68** / 0.71 |

라벨러 피드백에서 "직장인인데 일정이 수업"(5건)을 고쳤고, "중복"으로 보인 6쌍은 의도된 반례 쌍이라 안내문에 설명을 넣었다. 두 라벨러가 함께 설계자와 다른 5개는 다수결로 고쳤다. 라벨러가 엑셀로 다시 저장한 파일(행 전체가 따옴표로 묶임)을 읽도록 `agreement.py`도 고쳤다.

**강의 연결.** Ch03 p.26 Human evaluation의 단점 "Inter-annotator disagreement". 판단 보류 범주의 일치도(0.56, 0.33)가 가장 낮았다 — 사람도 갈리는 범주는 정답 자체가 불확실하다.

**교훈.** 설계자의 자체 점검은 **일관성**만 보여 주고 **타당성**은 보여 주지 못한다. 판정 후 일치도는 낙관적이므로 신뢰도는 판정 전 값으로 보고한다.

### #13 원문을 버리고 있었다 — 그래서 파서를 고칠 때마다 다시 돌려야 했다

```
# run_agent.py — 원문을 저장하고, 모델 없이 현재 파서로 재해석
act, why = parse(call_llm(SYSTEM_PROMPT_EN if a.lang == "en" else SYSTEM_PROMPT, build_user_message(r, a.lang), a))
raw = call_llm(SYSTEM_PROMPT_EN if a.lang == "en" else SYSTEM_PROMPT, build_user_message(r, a.lang), a)
act, why = parse(raw)
f.write(json.dumps({"case_id": r["case_id"], "run": run, "action": act, "reason": why, "raw": raw}) + "\n")
ap.add_argument("--reparse", ...)   # 저장된 원문(.jsonl)을 현재 파서로 다시 해석
ap.add_argument("--cases", ...)     # 시험 실행: S001,S039 ... 만
```

> **교훈.** 가장 비싼 것(모델 호출)의 결과는 **가공하기 전 원형 그대로** 남긴다. 본 실행 전에는 대표 사례 12개로 **시험 실행**을 하고 판단 근거를 사람이 읽는다. 무엇을 바꾸면 재실행이 필요한지(입력·프롬프트·도구)와 필요 없는지(파서·지표)를 미리 구분해 두고, 시험 실행 후에는 **동결**한다.

## 3. 실행기(하네스)의 시행착오

### #8 출력 길이 제한 150토큰

```
model=a.model, max_tokens=150, temperature=a.temperature,
model=a.model, max_tokens=300, temperature=a.temperature,
ap.add_argument("--timeout", type=float, default=90, ...)
ap.add_argument("--timeout", type=float, default=120, ...)
```

EXAONE은 실제로 응답을 ```` ```json ... ``` ```` 코드 블록으로 감싸고 근거를 2~3문장 썼다. 150토큰이면 긴 근거에서 JSON 끝이 잘려 형식 오류가 된다. 실행 전 예방 조치라 효과를 수치로 재지는 않았다.

> **강의 연결.** Ch04 p.46 출력 토큰이 비용과 지연을 정한다 — 그래서 줄이고 싶지만, 너무 줄이면 출력 형식이 깨진다.

### #9 파서가 너무 순진했다

```
# 수정 전: 첫 '{'부터 마지막 '}'까지를 한 덩어리로 (탐욕적 정규식)
m = re.search(r"\{.*\}", text, re.S)
d = json.loads(m.group(0))
# 수정 후: 괄호 짝으로 덩어리를 하나씩 꺼내 첫 번째 유효한 것, 코드 블록·끝 쉼표 허용
t = (text or "").replace("```json", "").replace("```", "")
for cand in _objects(t):                    # 문자열 안의 { } 는 건너뛰는 괄호 짝 탐색
    for c in (cand, re.sub(r",\s*([}\]])", r"\1", cand)):
        ... if isinstance(d, dict) and "action" in d: return d
```

| 모델 출력 | 수정 전 | 수정 후 |
|---|---|---|
| `Let me check. {"tool": ...} then {"action": "x"}` | PARSE_ERROR (두 JSON을 한 덩어리로) | 첫 번째 도구 호출 |
| `{"action": "gentle_prompt", "reason": "late night",}` | PARSE_ERROR | 정상 |
| `I think the user is on autopilot.` | PARSE_ERROR | PARSE_ERROR (그래야 맞다) |

> **강의 연결.** Ch04 p.37 Structured Output — "프롬프트로만 JSON을 요구하면 키가 빠지고 설명이 섞이고 JSON이 깨진다 → parse-and-retry 코드". 로컬 모델에는 Structured Outputs(제약 디코딩)가 없어서 이 단계를 우리가 직접 견고하게 만들어야 했다.

### #10 통계를 내려면 숫자 39개를 다시 적어야 했다

**증상.** 트랙 B 본 실행에서 `get_touch_log` 바로 다음에 `PARSE_ERROR`가 연달아 났다 (S068, S070, S071, S072 ...).

**원인.** 터치 기록은 간격·속도·방향 각 39개였다. 통계를 보려면 에이전트가 그 숫자를 JSON 안에 직접 다시 적어 `compute_stats`에 넘겨야 했다. 7B 모델은 `[33.1, 43.3, ...]`처럼 중간을 생략했고, 형식 오류 기회는 1번뿐이었다.

```
# run_agent_tools.py
x = np.array([float(t) for t in (args or {}).get("values", [])])
field = str(args.get("field", "")).strip()
if field or not isinstance(vals, list) or len(vals) < 2:   # 필드 이름이면 터치 기록에서 바로 계산
    vals = log.get(field if field in ("swipe_intervals_sec", "swipe_speed_px_per_s") else "swipe_intervals_sec", [])
- compute_stats: args {"values": [numbers]} -> n, mean, ...
- compute_stats: args {"field": "swipe_intervals_sec"} or {"field": "swipe_speed_px_per_s"} -> ...
if format_errors > 1:
if format_errors > 2:          # 안내 2번, 안내문에 field 방식 예시
```

**결과.** 재실행에서 `compute_stats`는 102번 모두 `field` 방식으로 호출됐고, 형식 오류는 4건(5%)으로 줄었다. 남은 4건은 `{"get_user_context": {}}`처럼 도구 이름을 키로 쓴 경우다.

> **강의 연결.** Ch04 p.44 **"Don't make the LLM compute what your program can"** — 계산·집계·조회는 코드가 하고 LLM은 이해와 판단만. Ch11 p.12 "Offload the burden from the model … Don't make the model fill arguments you already know", "Pass the intern test". Ch03 p.59·66 같은 모델도 하네스에 따라 점수가 3~20점 이상 달라진다.

> **교훈.** 도구 인자는 **모델이 이미 받은 데이터를 다시 옮겨 적게** 설계하면 안 된다. 모델은 "무엇을"(어느 필드)만 고르고, 데이터는 실행기가 들고 있어야 한다. 이 버그는 모델의 판단력이 아니라 **하네스의 결함**이 점수를 깎은 경우다.

## #11 "Usage concentration"을 주의력 점수로 읽었다

**증상.** EXAONE 트랙 A의 판단 근거 중 **72%**가 "concentration"을 언급했다. 예: *"The high concentration level and positive self-reported mood indicate intentional study behavior."* 이 값이 높으면 개입하지 않는 경향도 보였다(개입 안 함 예측 중앙값 0.76, 개입 예측 0.50).

**원인.** `usage_quadratic`은 "1시간 사용이 긴 세션 하나에 몰린 정도"다. 세션이 1개면 항상 1.0이고 세션 수와 상관 −0.67. 하지만 프롬프트의 이름표는 "Usage concentration (0-1)"뿐이었다.

```
# run_agent.py FIELDS_EN (트랙 B OBS도 동일)
("usage_quadratic", "Usage concentration (0-1)")
("usage_quadratic", "Session concentration (0-1): share of the hour's use packed into one long session,
                    1 = a single session; not a measure of attention")
```

| 같은 12개 사례, EXAONE 트랙 A | 수정 전 | 수정 후 |
|---|---|---|
| 정확도 | 0.58 | 0.58 |
| 같은 답을 고른 사례 | 12개 중 11개 | |
| 근거에 "concentration" 언급 | 78% | 33% |
| **"high concentration level"(주의력 해석)** | **44%** | **0%** |

수정 후 근거는 *"The high **session** concentration … indicate intentional engagement"*처럼 **사용 패턴**으로 읽었고, S066에서는 같은 값을 자동조종 쪽 근거로 썼다. **판단 결과는 거의 같았지만 판단 근거가 바뀌었다.** 정확도만 봤다면 찾지 못했을 결함이다.

> **강의 연결.** Ch11 p.12 "Write clear and detailed … parameter descriptions … what the output represents", Ch13 p.24 "Write descriptions well understood by LLMs". Ch04 p.39 Rule 7 — 하지 말 것만이 아니라 무엇인지를 말하라 ("not a measure of attention" + 무엇을 재는지).

## #12 CV의 방향만 알려 주고 눈금은 알려 주지 않았다

**증상.** "uniform swipe"라고 말한 근거 20건 중 **10건이 CV 0.75~2.0인 불규칙한 사례**였다. S030(CV 2.0)을 *"consistent reading pattern … uniform swipe intervals"*로 읽었다.

```
("swipe_interval_cv", "Swipe interval coefficient of variation (lower = more uniform)")
("swipe_interval_cv", "Swipe interval variability, CV (0 = perfectly regular; about 0.3 or lower = very regular;
                      about 1 or higher = irregular)")
("usage_sessions", "Number of sessions")
("usage_sessions", "Number of sessions in the last 60 minutes")
("avg_dwell_sec", "Average dwell per item (sec)")
("avg_dwell_sec", "Average time on each item (post or video) before swiping (sec)")
```

**지름길 위험은 없나.** "CV가 낮으면 자동조종"이라는 **판단**을 적으면 #1의 지름길(B3)을 가르치게 된다. 여기서는 **숫자를 읽는 법**(0.3과 1이 각각 무엇인지)만 적었다.

> **강의 연결.** Ch04 p.39 Rule 6 **"give numbers, not adjectives"** — "lower = more uniform"은 형용사형 설명이었고, 기준점 숫자가 필요했다. Ch04 p.45 "시각 대신 경과 시간을 알려 줘라"도 같은 원리다 — 우리가 일정에 "bedtime goal 24:00 (70 min past)"처럼 경과 시간을 미리 계산해 넣은 이유.

> **교훈 (#11, #12 공통).** 열 이름과 설명은 **사람에게 자명해도 모델에게는 아니다**. 판단 근거(`reason`)를 열별로 검색해 "이 열을 어떻게 읽었나"를 확인하는 것이 가장 빠른 점검 방법이었다. 설명은 ① 무엇을 재는지 ② 단위·기간 ③ 눈금의 기준점 ④ 흔한 오해(무엇이 아닌지)를 담는다.

## #15 출력 키 순서가 "행동 → 근거"였다

**발견.** 수정 계보를 정리하며 강의 자료를 다시 읽다가 찾았다. 실제 실행 결과에서 증상으로 드러난 것이 아니라 **설계 검토**에서 찾은 결함이다.

```
# run_agent.py SYSTEM_PROMPT_EN (트랙 B의 최종 결정 형식도 동일)
Output only one line of JSON: {"action": "<one of the three above>", "reason": "<1-2 sentences of reasoning>"}
Output only one line of JSON. Write the reason first, then the action that follows from it:
{"reason": "<1-2 sentences of reasoning>", "action": "<one of the three above>"}
```

**왜 결과가 바뀌나.** 자기회귀 생성에서는 토큰 $y_t$가 앞에서 생성된 $y_1 \cdots y_{t-1}$만 볼 수 있다. 수정 전 형식에서는 `"action"` 값이 근거보다 **먼저** 정해지고, `"reason"`은 이미 고른 행동을 정당화하는 문장으로 나온다. 수정 후에는 근거 토큰이 이미 맥락에 있는 상태에서 행동을 고르므로 판단이 근거에 실제로 의존할 수 있다(구조화된 출력 안의 chain-of-thought).

| 영향 | 내용 |
|---|---|
| 판단 | 근거를 먼저 정리한 뒤 고르므로, 판단 보류처럼 신호를 종합해야 하는 범주에서 차이가 날 수 있다. 효과는 본 실행에서 측정한다 |
| 판단 근거의 의미 | 수정 전 `reason`은 사후 정당화일 수 있었다. 이 문서의 #11·#12·#14에서 인용한 EXAONE 근거는 **수정 전 형식**의 실행이므로 "어떤 정보를 언급했는가"의 증거로만 쓴다 |
| 호환성 | 파서는 키 순서와 무관하게 읽으므로 이전 순서로 답해도 정상 처리된다. 예측 파일(`case_id, run, action, reason`) 형식은 그대로다 |

> **강의 연결.** Ch04 p.42 "Reason first, then act", p.43 "Schema key order = generation order — evidence → verdict, analysis → score, rationale → label".

> **교훈.** 출력 스키마의 **키 순서는 형식이 아니라 계산 순서**다. 판단을 요구하는 출력에서는 근거 → 결정 순서로 둔다. 또 이 결함은 실행 결과가 아니라 강의 자료를 다시 읽는 **설계 검토**로 찾았다. 실행 전 체크리스트로 점검할 수 있는 종류의 결함이다.

## #16 근거를 먼저 쓰게 하자 PARSE_ERROR가 10개 중 5개 났다 — 그리고 첫 진단은 틀렸다

**증상.** #15를 적용한 뒤 EXAONE 트랙 A로 10개 시험 실행을 하자 5개(S002, S003, S005, S006, S008)가 `PARSE_ERROR`였다. 모두 응답이 ```` ```json ```` 으로 시작했다.

**1차 진단: 출력 잘림 (틀림).** 근거가 맨 앞으로 오자 성공한 응답의 `reason`이 350~420자로 길어졌다(수정 전 중앙값 181자, 최대 301자). 근거가 길어져 한도에 걸리면 맨 끝의 `action`이 잘린다고 보고, 한도를 올리고 잘림을 표시하게 했다.

```
# run_agent.py, run_agent_tools.py
model=a.model, max_tokens=300, ...                    # 트랙 B는 600
MAX_TOKENS = 1000   # 근거를 먼저 쓰므로 한도에 걸리면 action이 잘린다. 잘림 방지용 여유
return r.choices[0].message.content
return r.choices[0].message.content, r.choices[0].finish_reason == "length"
if act == "PARSE_ERROR" and cut:
    why = "TRUNCATED (max_tokens): " + why                # .jsonl에도 "truncated" 저장
```

**원문으로 확인한 진짜 원인: 쉼표를 문자열 안에 넣는 습관.** 2차 시험 실행의 원문(`.jsonl`)을 보니 잘린 응답은 **0/10**이었고, 가장 긴 근거도 약 110토큰이었다. 대신 EXAONE은 근거 문자열을 `.,"`로 끝내는 습관이 있었다.

```
# predictions pilot_A.jsonl, S003 원문 (2차 시험 실행)
{"reason": "High frequency of refreshing comments ... catching news updates quickly.,"  "action": "gentle_prompt"}
# S010: action이 객체 밖으로 나감
{"reason": "User is navigating to a new location ... at this moment.,"}
"action": "gentle_prompt"
```

쉼표가 따옴표 **안**에 있으니 두 키 사이의 구분 쉼표가 없어 JSON이 깨진다. 수정 전 형식에서는 `reason`이 마지막 값이라 `"... quickly.,"}`가 되어 **우연히 유효한 JSON**이었다. #15로 `reason` 뒤에 키가 하나 더 붙자, 원래 있던 습관이 형식 오류로 드러난 것이다.

**고친 방법: 깨진 JSON에서 행동과 근거를 복구.**

```
ACT_RE = re.compile(r'"action"\s*:\s*"\s*(no_intervention|gentle_prompt|ask_context)\s*"')
def salvage(t):        # JSON으로 읽히지 않을 때만. 정확한 행동 이름이 없으면(잘린 응답) 복구하지 않는다
    m = ACT_RE.search(t)
    # 1차(PR #17): 근거를 "...", "action" 패턴으로 찾음 → 쉼표가 문자열 안에 있으면 근거가 빈칸
    w = re.search(r'"reason"\s*:\s*"(.*)"\s*,\s*"action"', t, re.S)
    # 2차(PR #18): reason 키부터 action 키 앞까지를 근거로, 남은 따옴표·쉼표·괄호는 정리
    w = REASON_KEY.search(t); end = m.start() if m.start() > w.end() else len(t)
    return m.group(1), t[w.end():end].strip().rstrip('{}," \n').strip()
```

| EXAONE 트랙 A 시험 실행 10개 | 1차 (#15 직후) | 2차 (복구 1차) | 2차 재해석 (복구 2차) |
|---|---|---|---|
| PARSE_ERROR | 5 | 0 | 0 |
| 근거가 빈 응답 | — | 3 (S003·S004·S010) | 0 |
| 엄격한 JSON 기준 형식 오류 | (원문 미확인) | 3 / 10 | |
| 토큰 한도에 걸린 응답 | (원문 미확인) | 0 / 10 | |

2차 재해석은 모델을 다시 부르지 않고 저장된 원문을 `--reparse`로 읽은 것이다(#13). 1차 시험 실행의 원문은 확인하지 못해, 그 5건이 모두 같은 원인이었는지는 확정할 수 없다. 2차의 표본이 같은 습관을 3/10 보였으므로 같은 원인이었을 가능성이 높다.

**왜 복구가 정당한가.** 복구는 `"action"` 키 바로 뒤에 **세 행동 이름 중 하나가 정확히** 있을 때만 한다. 모델이 분명히 고른 행동을 구두점 하나 때문에 버리면, 판단력이 아니라 형식 습관을 채점하게 된다. 반대로 행동이 없는 응답(잘린 응답, 설명만 있는 응답)은 여전히 `PARSE_ERROR`다.

> **강의 연결.** Ch04 p.37 Structured Output — 프롬프트로만 JSON을 요구하면 깨지고, 그래서 파싱·재시도 코드가 필요하다. 제약 디코딩(Structured Outputs)을 쓸 수 있으면 이 문제 자체가 사라진다. Ch03 p.59·66 하네스 의존성 — 키 순서 하나를 바꿨더니 같은 모델의 유효 응답이 50%로 떨어졌다. 고치지 않았다면 #15의 "효과"로 잘못 보고했을 것이다.

> **교훈.** ① 출력 형식을 바꾸면 **그 모델의 형식 습관과 새로 부딪힌다**. 형식 변경 하나에도 시험 실행이 필요하다. ② 1차 진단은 CSV에 남은 앞 200자만 보고 한 추측이었고 틀렸다. **원문을 먼저 봤다면** 한 번에 찾았을 것이다(#13의 교훈을 다시 확인). 잘림 표시를 넣은 덕분에 이제 "잘렸는가"는 추측이 아니라 기록으로 답한다.

## #17 트랙 B: 도구 이름을 `action` 키에 썼다

**증상.** 트랙 B 시험 실행 10개 중 S005의 최종 행동이 `compute_stats`로 기록됐다. 세 행동 중 하나가 아니므로 무효 응답이다. S002도 첫 시도에서 같은 형식을 썼다(그때는 JSON이 깨져 형식 오류 안내를 받음).

```
# pilot_B.jsonl, S005 원문
{"reason": "Swipe intervals show significant variability ... compute stats to confirm regularity pattern",
 "action": "compute_stats", "args": {"field": "swipe_intervals_sec"}}
```

**원인.** 최종 결정 형식이 `{"reason", "action"}`이 되자, 모델은 근거를 쓰고 나서 "다음 할 일"을 `action`에 적었다. 그 할 일이 결정이 아니라 도구 호출이었다. 도구 호출 형식 `{"tool", "args"}`와 결정 형식이 키 이름으로만 구분되는데, "action"이라는 이름이 두 경우 모두에 자연스럽게 읽힌다.

```
# run_agent_tools.py run_case
if isinstance(d, dict) and str(d.get("action", "")).strip() in TOOLS:   # 도구 이름을 action에 쓴 경우: 도구 호출로 본다
    d = {"tool": str(d["action"]).strip(), "args": d.get("args", {})}
# parse: 깨진 JSON 속의 도구 호출도 도구 이름(과 field)만 정확히 일치하면 살린다
TOOL_RE = re.compile(r'"(?:tool|action)"\s*:\s*"\s*(get_touch_log|compute_stats|get_user_context)\s*"' ...)
```

**결과.** 시험 실행 원문을 새 파서로 다시 읽으면 S005와 S002의 해당 응답은 모두 `compute_stats` 도구 호출로 해석된다. 대화가 이어지는 효과는 본 실행에서 확인한다.

**같이 넣은 예방 조치.** 본 실행 전에 형식 실수 목록을 직접 실행기에 넣어 점검했고, 아직 나오지 않았지만 의도가 분명한 형식도 받게 했다. ① 대소문자·공백만 다른 행동 이름(`Gentle_prompt`, `gentle prompt`)은 표준 이름으로 바꾼다. ② 작은따옴표 dict도 읽는다. ③ 트랙 B에서 도구 이름을 키로 쓴 호출(`{"get_user_context": {}}`, #10에서 4건 남았던 형식)과 함수 호출 API 형식(`{"name": ..., "arguments": ...}`)을 도구 호출로 받는다. ④ 트랙 B에서 세 행동 밖의 값(`"maybe"`)은 바로 무효로 끝내지 않고 형식 오류로 보고 다시 묻는다(트랙 A는 재시도가 없으므로 무효로 채점). `"gentle_prompt or ask_context"`처럼 둘 중 무엇인지 모호한 값은 어느 트랙에서도 복구하지 않는다.

**프롬프트는 왜 고치지 않았나.** "도구는 `tool` 키로만"이라고 덧붙일 수도 있었다. 하지만 본 실행 직전에 프롬프트를 또 바꾸면 #11·#12·#15와 효과가 더 섞인다. 모델의 의도가 분명한 경우라 실행기에서 받아 주는 쪽을 택했다.

> **강의 연결.** Ch11 p.12 "Principle of least surprise", 명확한 이름 — 결정 키와 도구 호출이 헷갈리지 않게 이름을 정해야 했다. Ch11 p.7 모델이 학습한 도구 형식을 쓰면 이런 혼동이 줄어든다. 우리는 모든 모델에 같은 조건을 주려고 JSON 한 줄 프로토콜을 써서 직접 다뤄야 했다(#9·#10과 같은 대가).

> **교훈.** 출력 스키마를 바꾸면 **다른 스키마와의 경계**도 다시 봐야 한다. #15는 결정 형식만 바꿨지만, 그 결과 도구 호출 형식과의 구분이 흐려졌다. 결정 값은 허용 목록으로 검사하고, 목록 밖의 값이 "다른 형식의 정상 응답"인지 먼저 확인한다.

## #18 트랙 A만 행동 정의가 달랐다 — "확인"과 "묻기"가 겹쳤다

**발견.** "이제 설계 결함은 없는가"라는 질문에 답하려고 두 실행기의 프롬프트를 줄 단위로 비교하다 찾았다. 실행 결과가 아니라 **점검으로** 찾은 두 번째 결함이다(#15 다음).

```
# run_agent.py SYSTEM_PROMPT_EN (트랙 A)
- gentle_prompt: ... show a small pop-up or TTS check-in about what the user wants to do. The user can always continue.
- gentle_prompt: ... show a small pop-up or TTS check-in. The user can always continue.
# 그대로: ask_context: ... Ask the user what they are trying to do first.
# 한국어도 같은 차이: "지금 하려는 일을 확인한다" → "확인한다", 그 밖의 표기 차이 2곳도 트랙 B와 같게
```

**왜 문제인가.**

- **두 행동이 겹쳤다.** 트랙 A의 `gentle_prompt`도 "하려는 일을 확인"하고, `ask_context`도 "하려는 일을 묻는다". 모델 입장에서는 흑백 전환이 붙느냐 말고는 차이가 없었다. 애매한 사례에서 `gentle_prompt`를 골라도 "어차피 묻는 것"이 된다. 시험 실행에서 두 모델 모두 트랙 A에서 `ask_context`를 한 번도 고르지 않았다(정답이 `ask_context`인 3개 × 2모델).
- **트랙 A와 B의 비교가 공정하지 않았다.** 트랙 B와 라벨링 지침에는 이 구절이 없다. #14의 "같은 모델 A 0.63 / B 0.36"은 행동 정의가 다른 상태에서 잰 값이다.
- **라벨러가 본 정의와도 달랐다.** 정답은 지침의 정의(구절 없음)로 정했다. 모델이 다른 정의를 보고 답했다면, 틀린 이유가 판단이 아니라 정의 차이일 수 있다.

**고친 방법.** 트랙 A의 행동 정의를 트랙 B와 글자 그대로 같게 했다. 지침의 정의와도 같다. 두 행동의 차이는 원래 설계대로 남는다: `gentle_prompt`는 자동조종이라고 **판단한 뒤** 마찰(흑백·확인)을 주고, `ask_context`는 판단하기 **전에** 묻는다.

**영향.** 이 구절이 `ask_context`를 고르지 않은 원인인지는 알 수 없다. 본 실행 4개는 모두 고친 정의로 돌리므로 모델 간·트랙 간 비교는 공정해진다. 이전 결과와의 비교에는 #11·#12·#15와 함께 또 하나의 변경이 섞인다.

### 후속 실험: 두 행동을 더 강하게 갈라 주면 묻기 시작하는가 (ablation)

구절을 뺀 뒤에도 `gentle_prompt`는 "확인(check-in)", `ask_context`는 "묻기"라 여전히 비슷해 보였다. 그래서 두 행동을 **판단 상태 · 마찰 · 질문 여부**로 가르고, 묻기의 쓰임과 부담을 한 문장 덧붙인 변형(`distinct`)을 만들어 본 실행 전에 비교했다.

```
# run_agent.py --prompt-variant distinct (본 실행 기본값 aligned와 다른 3줄)
- gentle_prompt: You judge that the user is on autopilot. Add light friction without blocking: ... nudge that points out the patter
n.
- ask_context: You cannot tell yet whether the use is intentional (...). Add no friction; first ask one neutral question ...
... When you cannot tell, asking is cheaper than a wrong decision, but asking when the answer is clear is also a burden.
```

**설계.** Qwen2.5-7B, 트랙 A, 같은 30개 사례(정답 `ask_context` 15 · `no_intervention` 8 · `gentle_prompt` 7), 각 1회. **판단 규칙은 결과를 보기 전에 정했다**: 묻기가 늘고 개입은 비슷하면 채택, 묻기는 비슷하고 개입만 늘면 기각, 1~2건 차이는 해석하지 않는다. 비교 스크립트는 `ablation.py`.

| Qwen 트랙 A, 30개 | aligned (본 실행) | distinct (분리안) |
|---|---|---|
| `ask_recall` (정답 AC 15개 중 물음) | 0 / 15 | 2 / 15 |
| `question_burden` (정답 NO 8개 중 물음) | 0 / 8 | 0 / 8 |
| `false_intervention` (정답 NO 8개 중 개입) | **0 / 8** | **3 / 8** |
| `detect_rate` (정답 GP 7개 중 개입) | 6 / 7 | 5 / 7 |
| 정확도 | 0.47 | 0.40 |

**결정: 채택하지 않았다.** 묻기는 2건 늘어 미리 정한 해석 기준(1~2건) 안이고, 불필요한 개입은 0 → 3건(S027, S037, S060)으로 늘었다. 분리안은 `gentle_prompt`를 "**가벼운** 마찰"로 설명하는데, 이것이 개입을 덜 부담스럽게 보이게 했을 수 있다.

**근거에서 보인 것.** 두 변형 모두 모델은 충돌하는 신호를 근거에 직접 쓰고도 한쪽을 골랐다. 예: S059 *"late at night, which could indicate mindless scrolling. However, ... some intentional use like saving posts"* → `gentle_prompt`. 정의를 분명히 해도 판단을 보류하지 않는다는 점에서, `ask_context`를 고르지 않는 것은 정의 문제라기보다 **모델의 성향**에 가깝다. 이 관찰은 근거를 먼저 쓰게 한 #15 덕분에 볼 수 있었다.

**이 실험이 말하지 못하는 것.** ① 각 변형을 1회만 돌렸고(temperature 0.3), 같은 프롬프트를 두 번 돌렸을 때 답이 얼마나 바뀌는지 모른다. 30개 중 12개가 변형에 따라 바뀌었는데, 그중 일부는 표본 잡음일 수 있다. ② #18 전 문구(`before18`)는 돌리지 않아서, 그 구절이 원인이었는지는 검증하지 않았다. ③ 분리안은 정의와 비용 문장을 함께 바꿔 둘의 효과를 나누지 못한다.

> **강의 연결.** Ch03 p.59·66 하네스 의존성 — 비교하려는 조건(트랙) 말고 다른 조건(행동 정의)이 같이 바뀌면 차이를 해석할 수 없다. Ch03 p.26 Human evaluation — 모델이 받는 과제 정의와 라벨러가 받는 정의가 같아야 정답과 비교할 수 있다. Ch04 p.39 Rule 7 명확한 지시 — 선택지의 정의가 서로 겹치지 않아야 한다. Ch03 p.46~47 반복 시행 — 1회 비교로는 변형의 효과와 표본 잡음을 나누지 못한다. Ch03 p.40 Goodhart — 묻기(`ask_recall`) 하나만 올리려다 불필요 개입이 늘었다. 지표를 함께 봐야 하는 이유다.

> **교훈.** ① 설계상 "더 맞아 보이는" 수정도 부작용이 있을 수 있다. 분리안은 정의로서는 더 분명했지만 개입을 늘렸다. 실험 없이 채택했다면 6~7시간의 본 실행이 끝난 뒤에야 알았을 것이다. ② 판단 규칙을 **결과를 보기 전에** 정해 두면, 결과를 본 뒤 마음에 드는 해석을 고르는 일을 막을 수 있다. ③ 같은 과제를 여러 실행기·여러 언어·라벨링 지침에 **따로 적으면** 조금씩 어긋난다. 행동 정의처럼 비교의 기준이 되는 문장은 한 곳에 두고 모든 곳이 그것을 가져다 쓰게 하거나, 최소한 **글자 단위 비교를 자동 검사**로 둔다. 선택지 정의는 서로 겹치지 않는지도 따로 읽는다.

## 4. 고치지 않은 것

### #14 트랙 B: 균일한 스와이프를 "집중"으로 읽고, 맥락은 40%만 확인했다

설명 수정 전, 하네스 수정 후의 결과다.

| EXAONE 7.8B | 정확도 | 탐지율 | 불필요 개입 | 맥락 확인율 |
|---|---|---|---|---|
| 트랙 A (정보를 한 번에 받음) | 0.63 | 0.79 | 0.25 | — (항상 있음) |
| 트랙 B (도구로 스스로 모음) | **0.36** | **0.12** | 0.06 | 0.40 |

```
# predictions_exaone_B.jsonl, S039 (밤 11:50, 알림으로 진입해 쇼츠 412개 연속 재생) — 실제 대화
a: {"tool": "get_touch_log", "args": {}}
u: Tool result: {"swipe_intervals_sec": [6.8, 6.4, 5.9, 6.1, ...], ...}
a: {"tool": "compute_stats", "args": {"field": "swipe_intervals_sec"}}
u: Tool result: {"n": 40, "mean": 6.01, "median": 5.95, "std": 0.74, "cv": 0.123, ...}
a: {"tool": "get_user_context", "args": {}}
u: Tool result: {... "recent_context": "Entry: tapped a recommended-video notification | In app: played 412 short videos ..."}
a: {"action": "no_intervention", "reason": "The user's swipe patterns show consistent engagement typical of intentional viewing
..."}
```

피치 덱의 "균일한 스와이프 = 좀비 스크롤"과 정반대로 읽었다. 맥락까지 확인한 뒤에도 처음 본 터치 통계의 인상을 따랐다. **이것은 고치지 않는다.** #11·#12는 벤치마크가 **잘못되거나 모호한 설명**을 준 결함이지만, 여기서는 설명이 정확한데 **모델이 다르게 해석**했다. "받은 데이터를 맥락과 함께 올바르게 해석하는가"가 바로 트랙 B가 재려는 능력이다.

> **강의 연결.** Ch13 p.4~5·16~18 **AI Workflow vs AI Agent** — 트랙 A는 정해진 정보 흐름(워크플로)에, 트랙 B는 스스로 도구를 고르는 에이전트에 가깝다. 강의의 권고 "First, try AI Workflows"가 작은 모델에서는 그대로 맞았다. Ch13 p.22 "필요한 정보를 받지 못하면 실패한다" — 트랙 B는 정보를 **받을 수 있었는데 고르지 않았다**. Ch03 p.46 에이전트 벤치마크는 답 하나가 아니라 궤적을 채점한다 — 그래서 `context_check_rate`를 따로 잰다.
> **Ch05 p.2~4 근거 없는 생성(deviation)** — 외부 정보가 없으면 LLM은 자기 사전 지식으로 답한다. 맥락을 확인하지 않은 사례의 EXAONE은 "일정한 스와이프 = 집중"이라는 통념으로 판단했다. **Ch05 p.50 Agentic RAG** — "LLM이 필요한 정보를 스스로 찾게 한다"가 바로 `get_user_context`이고, 작은 모델은 그것을 40%만 찾았다. Ch13 p.25~26은 이 때문에 **항상 넣어 주는 정보(RAG) + 필요할 때 찾는 정보(agentic retrieval)의 하이브리드**를 권한다. Phase 3에서 이 벤치마크로 에이전트를 만든다면 선언 목표와 관찰 기록은 처음부터 넣어 주고, 터치 기록만 도구로 찾게 하는 설계가 첫 번째 개선 후보다.

# 5. 통찰: 에이전트를 만들 때 확인할 것

## 5.1 코드 수준

| 확인할 것 | 구체적인 방법 | 근거 |
|---|---|---|
| 도구 인자에 모델이 옮겨 적을 데이터가 있는가 | 모델은 이름·ID·필드만 고르고, 실제 데이터는 실행기가 들고 있게 한다 ( `{"field": "swipe_intervals_sec"}` ) | #10 · Ch04 p.44, Ch11 p.12 |
| 출력 파서가 흔한 형식 실수를 견디는가 | 코드 블록, 앞뒤 설명, JSON 여러 개, 끝 쉼표를 테스트 케이스로 만든다. 설명만 있는 응답은 여전히 오류여야 한다. 가능하면 Structured Outputs를 쓴다 | #9 · Ch04 p.37 |
| 형식 오류 시 기회와 안내가 충분한가 | 재시도 횟수, 안내 문구에 올바른 예시 포함. 형식 오류율을 지표로 따로 잰다 | #10 |
| 출력 토큰 한도가 형식을 자르지 않는가 | 모델의 실제 출력 길이(코드 블록 포함)를 보고 여유를 둔다. 한도에 걸린 응답은 `finish_reason` 으로 기록해 추측하지 않는다 | #8·#16 · Ch04 p.46 |
| 모델 원문을 저장하는가 | 가공 전 원문을 `.jsonl` 로. 파서·지표를 고쳐도 재실행 없이 재채점 | #13 |
| 입력 열마다 설명이 충분한가 | 무엇을 재는지, 단위·기간, 눈금 기준점, 흔한 오해. "인턴 테스트": 이 설명만으로 사람이 정확히 읽을 수 있나 | #11·#12 · Ch11 p.12 |
| 출력 스키마의 키 순서 | 근거 → 결정 순서로. 결정이 근거에 실제로 의존하게 한다. 바꾼 뒤에는 반드시 다시 시험 실행한다 | #15·#16 · Ch04 p.42 |
| 결정 형식과 도구 호출 형식이 헷갈리지 않는가 | 결정 값은 허용 목록으로 검사한다. 목록 밖의 값이 도구 이름이면 도구 호출로 받는다 | #17 · Ch11 p.12 |
| 모든 실행기와 지침의 과제 정의가 같은가 | 행동 정의를 트랙·언어·라벨링 지침마다 글자 단위로 비교한다. 선택지끼리 겹치는 표현이 없는지 읽는다 | #18 · Ch03 p.26 |
| 파서 복구가 판단을 지어내지 않는가 | 정확한 행동 이름이 있을 때만 복구한다. 엄격한 JSON 기준 형식 오류율도 따로 센다 | #16 |
| 계산은 코드가 했는가 | 경과 시간, 횟수, 통계는 미리 계산해 준다 ("bedtime goal 24:00 (70 min past)") | Ch04 p.45 |

## 5.2 개념 수준

| 확인할 것 | 구체적인 방법 | 근거 |
|---|---|---|
| 이 데이터를 맥락 없이 풀 수 있는가 | 모든 열·단어에 대해 단일 규칙 정확도를 재고, 진단용 분류기(B4)로 문장 단서를 잰다. 지름길은 기준선으로 만든다 | #1·#2·#4 · Ch03 p.60 |
| 지표를 찍기로 이길 수 있는가 | 상수 전략(항상 같은 답)으로 모든 지표를 계산해 본다. 가중치는 민감도 분석으로 보고 | #3 · Ch03 p.40 |
| 정답이 설계자만의 판단인가 | 독립 라벨러, 판정 전 kappa로 신뢰도 보고, 범주별 일치도 확인 | #7 · Ch03 p.26 |
| 입력이 판단을 대신해 주지 않는가 | 요약("검색 없음")이 아니라 관찰("검색 0", 모든 사례에 같은 칸) | #4 · Ch13 p.22 |
| 결함과 측정 결과를 구분하는가 | 벤치마크가 잘못된 정보를 줬으면 고친다(#11·#12). 정보는 옳은데 모델이 틀렸으면 기록한다(#14) | #14 |
| 점수뿐 아니라 근거를 읽는가 | 설명 수정 후 정확도는 같았지만 근거가 바뀌었다. 판단 근거를 열별로 검색한다 | #11 · Ch04 p.66 |
| 워크플로로 충분한가, 에이전트가 필요한가 | 같은 모델로 둘 다 재 본다. 작은 모델은 정보를 스스로 고르게 하면 크게 떨어졌다(0.63 → 0.36) | #14 · Ch13 p.17~18 |
| 꼭 필요한 정보를 넣어 주는가, 찾게 하는가 | 판단에 항상 필요한 정보(선언 목표, 관찰 기록)는 처음부터 넣고, 비싸거나 가끔 필요한 정보(터치 기록)만 도구로 찾게 한다. 찾게 했다면 실제로 찾았는지 (`context_check_rate`)를 잰다 | #14 · Ch05 p.50, Ch13 p.25~26 |
| 프롬프트 변경을 비교 실험으로 확인했는가 | 같은 사례로 변경 전후를 비교하고(ablation), 판단 규칙과 해석 기준(몇 건 차이부터 의미가 있나)을 결과 전에 정한다. 주 지표 하나가 아니라 부작용 지표를 함께 본다 | #18 · Ch03 p.40·46 |
| 원인을 원문으로 확인했는가 | CSV의 요약이나 앞부분만 보고 추측하지 않는다. #16의 첫 진단(잘림)은 틀렸고, 원문을 보자 바로 진짜 원인(쉼표 습관)이 보였다 | #13·#16 |
| 실제 모델로 일찍 돌려 봤는가 | mock과 가짜 응답은 형식만 확인한다. 대표 사례로 시험 실행 → 근거 확인 → 동결 → 본 실행 | #10~#13 |

## 6. 강의 내용과의 대응

| 강의 (슬라이드) | 내용 | 이 프로젝트에서 겪은 일 |
|---|---|---|
| Ch03 p.26 Human evaluation | 평가자 간 불일치, 재현성 | #7 라벨러 2명, kappa 0.68~0.75, 판단 보류 범주가 가장 불일치 |
| Ch03 p.29~30, p.60 Contamination · Reward hacking | 과제 대신 허점을 푸는 모델 | #1·#2·#4 CV·체류·부정어 지름길, B3·B4로 측정 |
| Ch03 p.40 Goodhart의 법칙 | 측정치가 목표가 되면 망가진다 | #3 항상 묻기가 비용 점수 1위, 여러 지표를 함께 성공 기준으로 |
| Ch03 p.46~47 Agentic benchmark | 궤적 채점, 반복 시행, pass^k | 트랙 B 도구 사용 지표, 3회 실행 일관성 |
| Ch03 p.59, p.66 하네스 의존성 | 같은 모델도 하네스에 따라 점수가 크게 바뀜 | #10 숫자 복사 도구 → 연속 형식 오류, #14 같은 모델 A 0.63 / B 0.36, #16 키 순서 변경 → 유효 응답 50%, #18 트랙마다 다른 행동 정의 |
| Ch04 p.37 Structured Output | 프롬프트로만 JSON 요구 → 깨짐 → 파싱·재시도 | #8·#9 출력 한도, 견고한 파서, #16 깨진 JSON에서 행동·근거 복구 |
| Ch04 p.39 Rule 6·7 | 형용사 대신 숫자, 하지 말 것 대신 할 것 | #11 "not a measure of attention" + 정의, #12 CV 기준점 0 / 0.3 / 1 |
| Ch04 p.42~43 Reason first | 스키마 키 순서 = 생성 순서 | #15 action → reason을 reason → action으로 변경 (부작용 #16·#17) |
| Ch04 p.44~45 계산은 코드로 | 조회·계산·경과 시간은 프로그램이 | #10 `field` 인자, 일정의 "(70 min past)" |
| Ch04 p.66 Build evals | 프롬프트 행동을 재는 평가를 만들어라 | #13 시험 실행, 판단 근거 열별 검색 |
| Ch05 p.2~4 RAG의 필요성 | 외부 근거가 없으면 사전 지식으로 답하다 빗나감(deviation) | #14 맥락을 확인하지 않은 판단은 "일정 = 집중"이라는 통념을 따름 |
| Ch05 p.32~35 Keyword vs Vector search | 단어 일치와 의미 유사도의 차이 | #4 B4(단어 빈도 분류기)로 문장 표면 단서를 측정, 의미 단어만 남김 |
| Ch05 p.50 Agentic RAG (+ Ch13 p.25~26 하이브리드) | LLM이 스스로 정보를 찾게 함, RAG와 섞는 전략 | #14 트랙 B 맥락 확인율 40%, Phase 3 개선 방향으로 하이브리드 제안 |
| Ch11 p.7 도구 프롬프트 형식 | 모델이 학습한 도구 형식을 따르는 편이 정확 | 모든 모델에 같은 조건을 주려고 JSON 한 줄 프로토콜 사용 → 형식 오류를 직접 다뤄야 했음(#9·#10) |
| Ch11 p.12 함수 정의 모범 사례 | 명확한 설명, 최소 놀람, 인턴 테스트, 모델 부담 줄이기 | #10 도구 인자, #11·#12 열 설명, #17 결정 키와 도구 호출의 혼동 |
| Ch12 Memory | 사용자 메타데이터, 의미 기억 | #6 사용자 상태, 평소 사용량(실제 값)을 사용자 기억처럼 제공 |
| Ch13 p.4~5, 16~18 Workflow vs Agent | 먼저 워크플로를 시도하라 | #14 작은 모델에서 트랙 A(워크플로형) > 트랙 B(에이전트형) |
| Ch13 p.19~24 Context engineering | 필요한 정보를 적절히, 도구 설명은 명확히 | #4 관찰 기록, #14 맥락 확인율 40%, #11·#12 설명 |

**정직하게 남겨 둘 한계**

- 설계와 실행기를 만든 쪽(Claude)은 실제 모델을 직접 돌릴 수 없는 환경에서 작업했다. 형식 오류(#10)와 해석 문제(#11·#12)가 사용자의 실제 실행에서야 드러난 가장 큰 이유다. 결과적으로 "실제 모델로 일찍 시험 실행"이 가장 비용 대비 효과가 큰 점검이었다.
- #11의 효과는 12개 사례 시험 실행으로만 확인했다. 81개 전체의 수정 후 결과는 현재 실행 중이다.
- #18의 ablation은 Qwen 한 모델, 30개 사례, 각 1회다. 같은 프롬프트의 반복 실행 간 차이를 재지 않아 효과와 잡음을 구분하지 못한다. 결정은 미리 정한 기준(불필요 개입 증가)에 따른 것이고, 분리안이 "효과가 없다"고 증명한 것은 아니다.
- #14의 트랙 A 0.63 / B 0.36은 행동 정의가 트랙마다 달랐던 상태(#18)에서 잰 값이다.
- #14의 트랙 B 결과는 설명 수정(#11·#12) 전 실행이다. 수정 후 결과가 달라질 수 있다.
- 이 문서에서 인용한 EXAONE의 판단 근거는 모두 #15 수정 전(행동 → 근거) 실행이라 사후 정당화일 수 있다. 근거 분석은 "모델이 어떤 정보를 언급했는가"의 증거로만 쓴다.
- #16·#17의 복구 때문에 `invalid_rate`는 "행동을 알아볼 수 없는 응답"의 비율이다. 엄격한 JSON 기준 형식 오류율은 이보다 높다(EXAONE 트랙 A 시험 실행 3/10).
- #15 이후 트랙 B 시험 실행 10개에서 EXAONE은 `get_user_context`를 한 번도 부르지 않았고(맥락 확인 0/10), 불규칙한 간격을 자동조종, 일정한 간격을 집중으로 읽었다. #14와 같은 경향이지만 10개라 단정하지 않는다.
- #15의 효과는 아직 재지 않았다. 지금 실행 중인 본 실행(근거 → 행동)과 #14의 수정 전 결과를 비교할 수는 있지만, #11·#12 설명 수정도 함께 바뀌어 효과를 분리하지 못한다.